-

Software Requirements Specification

for

Network Guardian

Version 1.1 approved

Prepared by Ryan Hayden, Uriah Stover, Tobias Hill, Chloe Thacker

West Virginia University Institute of Technology

9/30/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction **4**
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 5
2. Overall Description **6**
2.1 Product Perspective 6
2.2 Product Functions 7
2.3 User Classes and Characteristics 7
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 8
2.6 User Documentation 9
2.7 Assumptions and Dependencies 9
3. External Interface Requirements **9**
3.1 User Interfaces 9
3.1.1 WebUI 9
3.2 Hardware Interfaces 10
3.2.1 LAN to Broker Service 10
3.2.2 WebUI to Laptop 10
3.2.3 IoT Devices to ML Model 11
3.3 Software Interfaces 11
3.3.1 MQTT Protocol 11
3.3.2 Software on the Server 11
3.3.3 Test Bed Server and IoT Devices 11
3.3.4 WebUI 12
3.4 Communications Interfaces 12
4. System Features **12**
4.1 Network Traffic Monitoring 12
4.2 Feature Extraction 12
4.3 Machine Learning-Based Intrusion Detection 13
4.4 Model Adaptation/Calibration 13
4.5 Detection Logging and Storage 14
4.6 Web-Based Dashboard 14
4.7 Evaluation and Performance Modeling 15
5. Other Nonfunctional Requirements **16**
5.1 Performance Requirements 16
5.2 Safety Requirements 16
5.3 Security Requirements 17

5.4 Software Quality Attributes 17
5.5 Business Rules 17
6. Other Requirements **17**

Revision History

Name	Date	Reason For Changes	Version
All	9/25	Initial Version	1.0
All	9/28	Addressed Mentor's Comments	1.1

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification document is to describe Network Guardian, an open-source tool that detects anomalies in network traffic on an IoT (Internet of Things) network. This is for Version 1.0 and covers the entirety of the software product.

1.2 Document Conventions

This document was created using the IEEE (Institute of Electrical and Electronics Engineers) template provided by Dr. Sakkaff in CS/EE 480. The title page uses an Arial typeface. The body of the document is in Times New Roman typeface.

1.3 Intended Audience and Reading Suggestions

This document is intended for Network Guardian software developers and any relevant WVU Tech instructors. This document should be used to advise developers on choices made during development and what features are included in the software. CS 480 instructors are expected to review the document to understand Network Guardian's project. Others may review the document out of their own interest or to give advice.

It is recommended to read this document starting with the *Introduction*, followed by the *Overall Description*, and then *System Features*. This will allow for the best understanding of the document's contents. The rest can be read in any order for best understanding.

1.4 Product Scope

Network Guardian is a network packet analyzer that can adapt to the network it's on to detect malicious or abnormal traffic. It is an open-source project designed for homes and small businesses. It is made with IoT in mind, so that it can work with smart devices along with computers and servers. There are multiple attack types we plan to test and train with initially. Long term, the goal is to have the system be adaptable and be able to detect and classify types of attack.

SCOPE NOTE: The initial project is intended as a detection tool and doesn't filter or block flagged traffic.

The project wishes to answer two general questions:

A. How effective is our machine learning model at adapting to unseen IoT LAN (Local Area Network) network environments and configurations?
B. How effective is our ML (Machine Learning) model at improving performance from online adaptation to network-specific patterns?

This will be analyzed through repeated testing of different numbers or types of devices, traffic rates, MQTT frequencies, and normal communication behavior.

1.5 References

[1] “AWID - Aegean Wi-Fi Intrusion Dataset,” Aegean.gr, 2024. [Online]. Available: https://icsdweb.aegean.gr/awid/ (accessed: Sept. 25, 2026).
[2] F. Cerasuolo, G. Bovenzi, C. Marescalco, F. Cirillo, D. Ciuonzo, and A. Pescapè, "Adaptive Intrusion Detection Systems: Class Incremental Learning for IoT Emerging Threats," in *2023 IEEE International Conference on Big Data (BigData)*, Sorrento, Italy, 2023, pp. 3547–3555, doi: 10.1109/BigData59044.2023.10386129.
[3] Pandas, “pandas documentation — pandas 1.0.1 documentation,” pandas.pydata.org, 2024. [Online]. Available: https://pandas.pydata.org/docs/ (accessed: Sept. 25, 2026).
[4] Scikit-Learn, “User guide: contents — scikit-learn 0.22.1 documentation,” Scikit-learn.org, 2019. [Online]. Available: https://scikit-learn.org/stable/user_guide.html (accessed: Sept. 25, 2026).
[5] Joblib Developers “Development — joblib 1.5.2 documentation,” Readthedocs.io, 2025. [Online]. Available: https://joblib.readthedocs.io/en/stable/developing.html (accessed: Sept. 25, 2026).
[6] M. A. Mahdi, "Secure and Efficient IoT Networks: An AI and ML-based Intrusion Detection System," 2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT), Vellore, India, 2024, pp. 1-6, doi: 10.1109/AIIoT58432.2024.10574789.
[7] B. Laxmaiah, V. Ramesh, P. Patel, M. Prashanthi, J.Deepika and R.BagavathiLakshmi, "AI Based Cyber security Solutions for Detecting and Preventing Threats in IoT Systems," 2025 2nd International Conference on Recent Trends in Electrical, Electronics and Computing Technologies (ICRTEECT), Warangal, India, 2025, pp. 1-6, doi: 10.1109/ICRTEECT67512.2025.11448925.
[8] Python Software Foundation, "Download Python," Python.org. [Online]. Available: https://www.python.org/downloads/. (accessed: Sept. 25, 2026).
[9] Raspberry Pi Ltd, "Raspberry Pi OS," Raspberry Pi Documentation. [Online]. Available: https://www.raspberrypi.com/documentation/computers/os.html. (accessed: Sept. 25, 2026).
[10] MQTT.org, "MQTT: The standard for IoT messaging," 2024. [Online]. Available: https://mqtt.org/. (accessed: Sept. 25, 2026).
[11] GitHub, Inc., "Quickstart for writing on GitHub," GitHub Docs. [Online]. Available: https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/quickstart-for-writing-on-github. (accessed: Sept. 25, 2026).
[12] Eclipse Foundation, "Eclipse Mosquitto: An open source MQTT broker," 2018. [Online]. Available: https://mosquitto.org/. (accessed: Sept. 25, 2026).
[13] Raspberry Pi Ltd., “Buy a Raspberry Pi 5,” Raspberry Pi, 2026. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5/. (accessed: Sept. 25, 2026).
[14] M. Grinberg, “Flask-SocketIO,” Flask-SocketIO Documentation, 2026. [Online]. Available: https://flask-socketio.readthedocs.io/en/latest/. (accessed: Sept. 25, 2026).
[15] SQLite, “SQLite: Small. Fast. Reliable. Choose any three,” SQLite. [Online]. Available: https://www.sqlite.org/. (accessed: Sept. 25, 2026).
[16] Linux Wireless, “hostapd: IEEE 802.11 AP, IEEE 802.1X/WPA/WPA2/WPA3/EAP/RADIUS authenticator,” Linux Wireless Documentation. [Online]. Available: https://wireless.docs.kernel.org/en/latest/en/users/documentation/hostapd.html. (accessed: Sept. 25, 2026).
[17] S. Kelley, “dnsmasq,” dnsmasq Documentation. [Online]. Available: https://thekelleys.org.uk/dnsmasq/doc.html. (accessed: Sept. 25, 2026).
[18] Arduino, “PubSubClient,” Arduino Documentation. [Online]. Available: https://docs.arduino.cc/libraries/pubsubclient/. (accessed: Sept. 25, 2026).
[19] B. Blanchon, “ArduinoJson,” ArduinoJson Documentation. [Online]. Available: https://arduinojson.org/. (accessed: Sept. 25, 2026).
[20] Cisco Meraki, “Wireless Fundamentals: Encryption and Authentication,” *Cisco Meraki Documentation*, 2026. [Online]. Available:

https://documentation.meraki.com/Wireless/Design_and_Configure/Architecture_and_Best_Practices/Wireless_Fundamentals:_Encryption_and_Authentication. (accessed: Sept. 25, 2026).
[21] IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, IEEE Std 802.11-2024, 2025.
[22] Federal Communications Commission, "Part 15—Radio Frequency Devices," *Code of Federal Regulations*, 47 C.F.R. § 15.5. [Online]. Available: https://www.ecfr.gov/current/title-47/chapter-I/subchapter-A/part-15. (accessed: Sept. 25, 2026).
[23] U.S. Congress, "18 U.S.C. § 1030—Fraud and related activity in connection with computers," *United States Code*, Title 18, § 1030. [Online]. Available: https://uscode.house.gov/view.xhtml?edition=prelim&req=granuleid%3AUSC-prelim-title18-section1030. (accessed: Sept. 25, 2026).

2. Overall Description

2.1 Product Perspective

Network Guardian aims to create a machine learning model to detect anomalous behavior across different IoT LAN network environments and configurations. We intend to create different AI (Artificial Intelligence) models to test their adaptability to different devices and attacks. This tool is not an extension of any product family and is not designed to replace existing systems. It is designed to integrate easily into the monitored network.

For the 2026 CS480 class, we intend to use our product to test a simulated environment that we will build to demonstrate how effective our ML models are based on the metrics we collect. A top-down view of the network and subsystems is shown below in Figure 1.

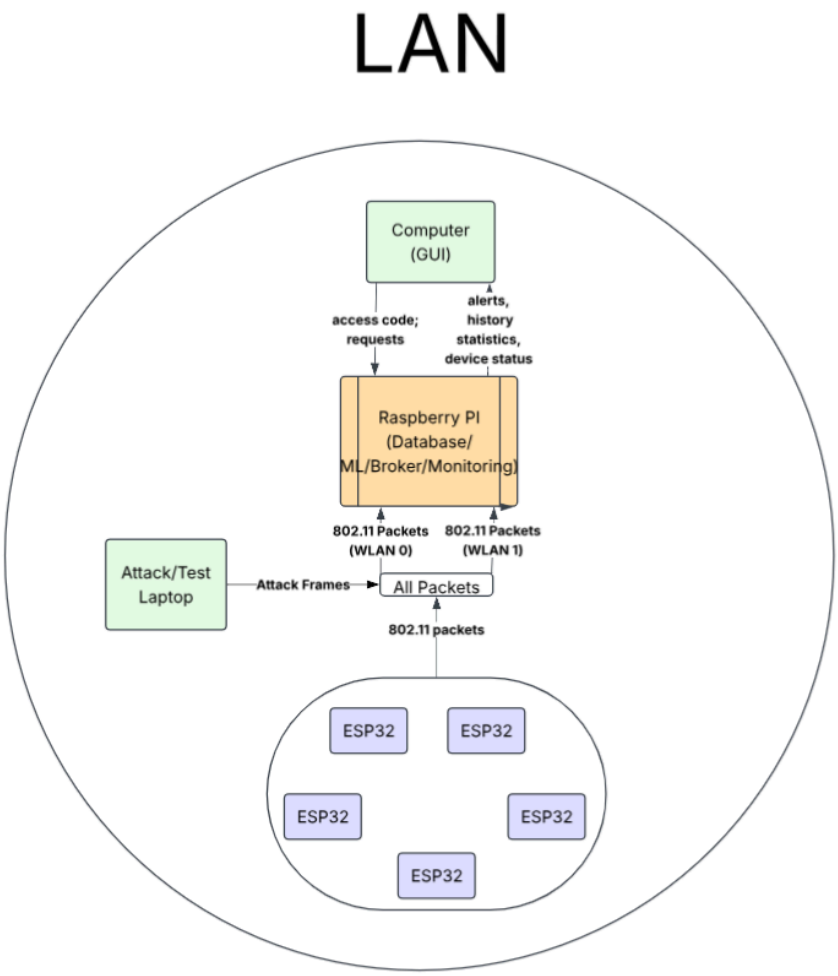


Figure 1: Top-down Network view

2.2 Product Functions

- Identify anomalies in network traffic
- Adapt to different network environments and configurations
- Classify attack types with a confidence score
- Display new findings when analyzed on the network
- Provide a history of previously flagged network traffic

2.3 User Classes and Characteristics

The primary user classes for this product are small businesses, homeowners, and individual users. Functionality would be the same for all three groups; however, the purpose behind their choice would differ. All groups would most likely use all the functions provided by our product since one function isn't better suited for another, and all are important to getting the full value of the model.

Small businesses would want to see if there's anything coming through that could potentially be harmful, especially if their businesses offer online services or store important information on their network. This

product would allow them to test their network regardless of devices connected at the time and see how safe their network is.

Homeowners wouldn't need to worry as much, as they will have all access to all general admin access. They wouldn't be storing or handling important information besides their own, but some might want to ensure their network is safe. It would also make it easy for them to set up and check since our product won't require much to get up and running on the network.

Individual users could use this for testing or for their own research purposes. They are likely to check the statistics of the model and its performance to draw conclusions or develop their own type of system that would focus on a particular area. The user shall have review, dismiss, delete, and recalibrate privileges.

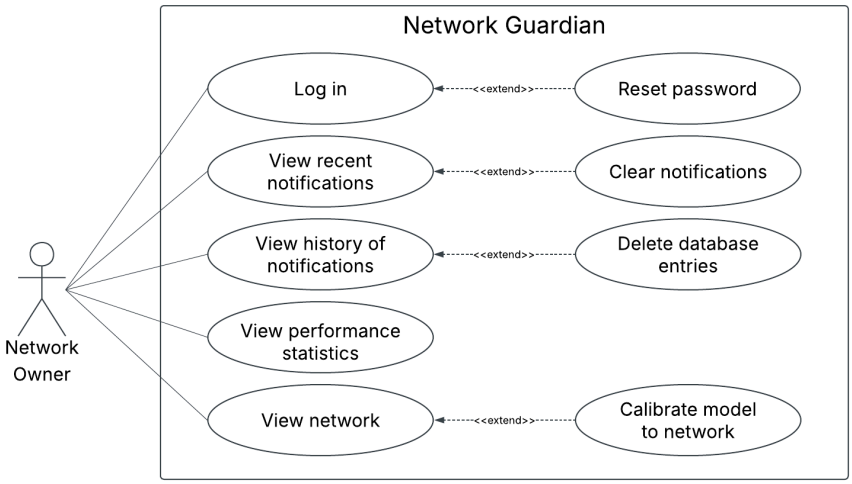


Figure 2: Use Case Diagram

2.4 Operating Environment

Any software developed or acquired should be able to operate on a Raspberry Pi 5 with 8 GB, which is not limited to the machine learning model, internet traffic capture software, database, web UI, and the MQTT broker service. The Raspberry Pi will be running a lightweight OS such as Raspberry Pi OS Lite (Bookworm, released 2023) or newer due to performance concerns. Any other software installed on the Raspberry Pi that does not support our project will be uninstalled or disabled.

2.5 Design and Implementation Constraints

Our system is designed to work locally on the network it's connected to, meaning everything we use must be stored and run on the hardware. The Raspberry Pi RAM and storage will be limited, so our software, database, and WebUI must be lightweight enough to run without causing the system's memory to overflow. The data must be processed as it comes in due to the limited RAM in the hardware. The Raspberry Pi acting as the access point will also cause some limitations. The traffic that comes through the network can't exceed what the Raspberry Pi can handle to prevent a bottleneck. The scalability of our product is somewhat limited based on the number of devices and traffic coming through the network. When we test our product, we must test it on an isolated network due to the simulated attacks it will be trained/tested on. The Raspberry Pi's

Wi-Fi chip doesn't allow us to get all the data we need, so we will need an external USB Wi-Fi adapter to capture the data transmitted between devices.

2.6 User Documentation

Upon the release of the software, a step-by-step installation guide will be provided with the prospect of installing the machine learning model on the network. In addition, a comprehensive user manual will be provided that demonstrates the entire infrastructure of the project. A GitHub README will be written to explain how the software works or technical details the user may want to know. The README will be made using the Markdown (.md) file format, as this is the primary standard. We will also write a quick start guide that will show the process to turn on Network Guardian. This will include a clear step-by-step setup that will allow individuals who have some technical experience to follow. This will be written in Word following the industry Microsoft Style Guide and exported as a PDF.

2.7 Assumptions and Dependencies

While developing our product, we are assuming the Raspberry Pi will have sufficient processing power to handle the software, database, and WebUI. We are also assuming that our components will all work as intended, functioning together to gather and analyze the data across the system. The product has multiple dependencies: the libraries we are using to train our model, the libraries used for the ESP-32s, and the libraries that will be used on the Raspberry Pi. Another dependency we have is the datasets used to train our models. If any of the concerns listed were not to perform the way we intend or change at the last minute, we would need to reconsider our strategy and decide on new ways to achieve our goal.

3. External Interface Requirements

3.1 User Interfaces

3.1.1 WebUI

Network Guardian will have a webpage for users. There will be sections for notifications, history, performance statistics, and network devices. The notification section will have the type of attack, a summary of packet data, and a dismiss alert feature. The history section will show a history of dismissed instances, and they will be retained for six months. The performance section will have the data and statistics of the performance of the ML model. The network devices section will show a list of connected devices and statistics on them.

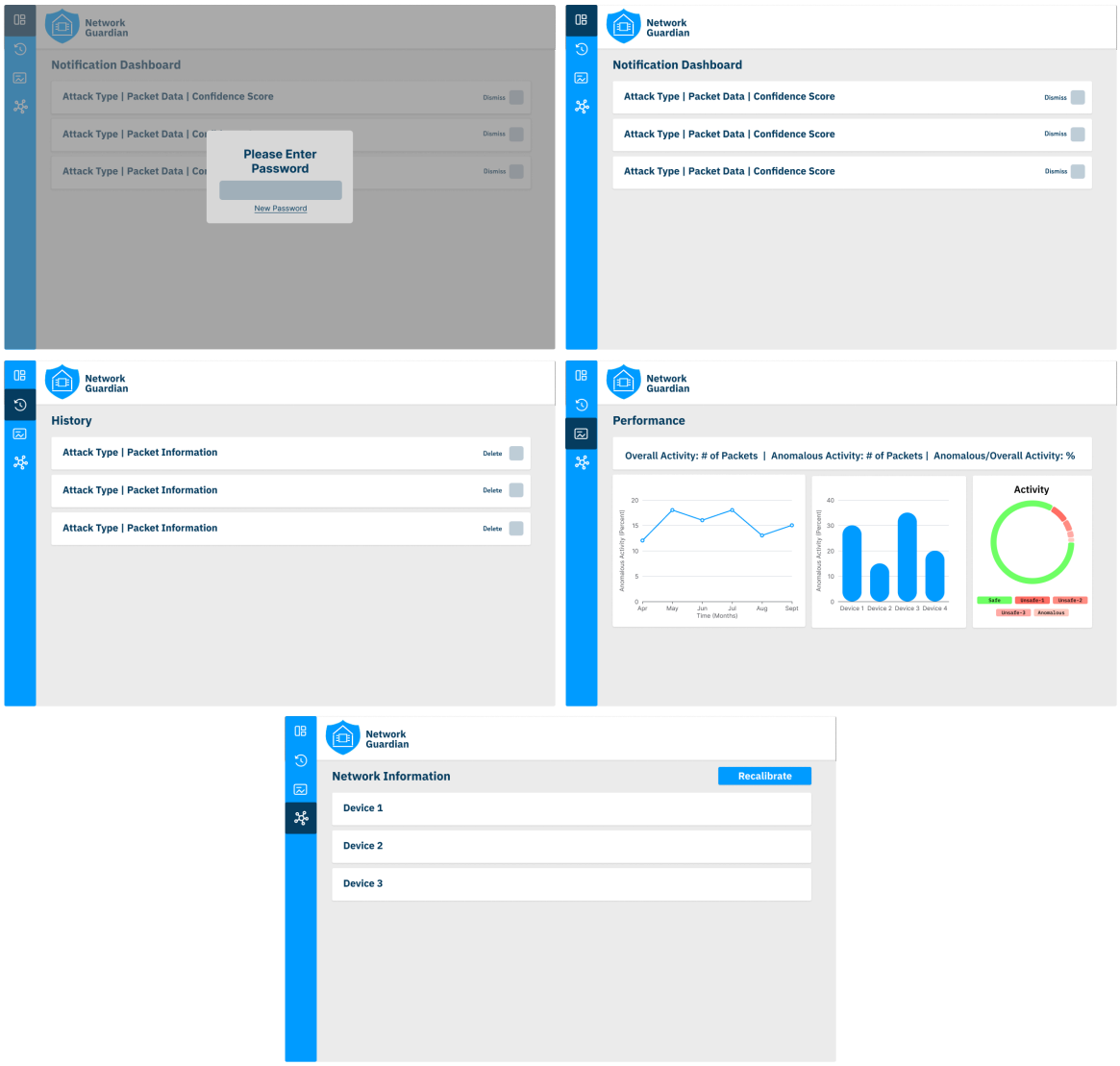


Figure 3: WebUI Sections

3.2 Hardware Interfaces

3.2.1 LAN to Broker Service

The connection between the IoT devices and the MQTT broker service, called Eclipse Mosquitto, will be through the wireless access point, which will be created on the Raspberry Pi's Wi-Fi chip (wlan0), Broadcom BCM2712. So the connection between the ESP-32s and the broker service will be through the ESP-32's Wi-Fi chip and the Raspberry Pi's Wi-Fi chip.

3.2.2 WebUI to Laptop

The connection between the Raspberry Pi and the Laptop will be made through the Ethernet port found on both. This will place the laptop on the LAN, which will allow the computer to connect to the IP address and port hosting the WebUI. The exact IP address will be determined later and will be documented in the user manual.

3.2.3 IoT Devices to ML Model

To be able to monitor the packets on the network, an external USB Wi-Fi adapter (wlan1) will be used in monitor mode to capture all of the packets on the network. The Wi-Fi adapter will act as a packet sniffer, which picks up the packets on the network and sends the packets to the machine learning model. The external USB Wi-Fi adapter will be attached to a direct USB Type-A connection to the Raspberry Pi's USB Type-A port.

3.3 Software Interfaces

3.3.1 MQTT Protocol

The MQTT protocol is a lightweight messaging protocol that is designed specifically for IoT communication. The devices use the protocol to communicate with a central server called a broker. We are planning to use Eclipse Mosquitto as the broker service. This protocol uses the publish/subscribe model.

3.3.2 Software on the Server

For the backend of this project, we will be using the newest or a new version of Python to program our system. The exact version of Python will be 3.13, as it was released on 10.07.24 and it passed its bugfix timeline. We will be using different Python libraries to train our ML model. Pandas to handle our data, mainly reading and loading the data from the dataset. Scikit-learn will be added to create models such as random forest and isolation forest. Joblib will allow us to save and load the model while training and retesting. We will be using the MQTT Broker version called Eclipse Mosquitto will be version 2.1.2-3 or the latest version for Linux that is available. We will be using version 3 or the latest version of Flask to host our WebUI, as it is a web server framework, and this deviates from the proposal, as it mentions using Django. We will also be using Flask's Flask-SocketIO for real-time WebSocket communication, and we will be using the latest version available. We will be using version 2.71 or the latest version of Scapy, which will be used to capture the packets. As mentioned in the proposal, we will be using SQL to store "abnormal packets" so that we can analyze how the ML model performs. We will use SQLite 3 as our database software, as this is the latest version.

3.3.3 Test Bed Server and IoT Devices

In addition to the software built for the Raspberry Pi, a hardware test bed will be built using ESP-32 development boards, which will simulate IoT device traffic on an IoT LAN. Multiple software scripts will be programmed to simulate different devices, such as but not limited to smart lamps, smart outlets, or smart thermostats. The Raspberry Pi will run as an access point for these devices. We will be using two different softwares to set up the Wi-Fi access point, the first is hostapd, which will create and manage the Wi-Fi access point on wlan0. The second is dnsmasq, which will set up the DHCP server for each of the embedded devices. The code written to the ESP32s will be written in the Arduino IDE with the latest version. The table describes the libraries we will be using, the version number, and the purpose of that library in our project.

Table 1: Libraries used on ESP-32

Library	Version	Purpose
Wi-Fi	Built-in	Wi-Fi connectivity
PubSubClient	2.8.0+	MQTT Client Protocol

ArduinoJson	6.21.0+	JSON serialization/deserialization
time.h	Built-in	Timing and timestamps

The computer will use an ALFA Network AWUS036ACM Wifi adapter to connect to the Raspberry Pi's access point to deliver different attack payloads. The types of attacks are ARP Spoofing, Deauthentication attack, and a replay attack. If time permits, we will add malformed and fuzzed traffic attack detection.

3.3.4 WebUI

The UI will be made such that a user can connect to the Raspberry Pi through an ethernet cable, which will place the laptop on the network isolated from the WVU or regular Wi-Fi networks. The laptop will be able to load the locally hosted WebUI simply by connecting to the IP address on port 8080 of the Raspberry Pi. The type of operating system for this part is inconsequential, as long as it is a modern operating system.

3.4 Communications Interfaces

The Wi-Fi access point will be used to connect the IoT devices, and this will use the IEEE 802.11b/g/n protocol. This will use WPA2-AES encryption. The Wi-Fi Monitor using the USB Wi-Fi adapter will use the 802.11 raw frames for capturing packets. MQTT will use TCP/IP over Wi-Fi for IoT pub/sub messaging. The Ethernet cables will use the Gigabit Ethernet protocol for WebUI management. HTTP will use TCP/IP over Ethernet for running WebUI requests. SQLite will be using local file I/O for database access. All of this software has yet to be tested on the test bed, and therefore the transmission rate of each has not been measured.

4. System Features

4.1 Network Traffic Monitoring

4.1.1 Description and Priority

Monitors the network traffic so that packets are sent to the Raspberry Pi and fed into the model for analysis. It should monitor all network traffic due to the antenna and network configuration. This is a high-priority feature as it is required for the project to work.

4.1.2 Stimulus/Response Sequences

Receives traffic from devices and sends it to the Raspberry Pi

4.1.3 Functional Requirements

NTM-01: The system shall capture all 802.11 frames on the access point's channel using wlan1 in monitor mode.

NTM-02: Data shall not be sent elsewhere within the system.

4.2 Feature Extraction

4.2.1 Description and Priority

The feature extraction will obtain the per-packet and time-windowed features from the captured data to be turned into numeric data. The per-packet fields are frame type/subtype, sequence number, RSSI, duration, inter-frame time delta, retry flag, and protected flag. The time-windowed fields are packet-per-second-window, unique source MACs per window, and duplicate sequence count per window. This is a high-priority feature since it needs to be able to translate the data into something the model can understand.

4.2.2 Stimulus/Response Sequences

As the data comes in, the features are extracted to be converted into numeric data. Once the extraction and conversion happen, the data is sent to the model to be analyzed.

4.2.3 Functional Requirements

FE-01: The system shall compute time-windowed features only within the current rolling window.

FE-02: The feature pipeline shall align live-capture columns to the trained model's expected column order before inference

4.3 Machine Learning-Based Intrusion Detection

4.3.1 Description and Priority

This model will take in the network traffic and analyze the packets. If the traffic isn't flagged, nothing happens, and the system continues as usual. If it's flagged by the model, the attack type is determined by the random forest model, the data is recorded, and stored in the database. The supervised model will be trained on the WVU Tech GPU workstations. The supervised model will be trained using the AWID2 dataset to give labels to attack types and serve as an initial baseline. It will later be used to classify attacks the unsupervised model flags. The unsupervised models will be trained with simulated data. If an attack type is unknown to the supervised model or can't be classified, it will be labeled as unclassified/unknown. This is the highest priority as it is the core of the project.

4.3.2 Stimulus/Response Sequences

The isolation forest model receives numeric input. That data is then processed through the isolation tree to determine if it should be flagged or not. Once that determination is made, the data is either recorded and stored or dropped entirely. If the data is being stored, the random forest model will analyze and determine the attack type. If an attack type can't be produced, it is labeled as unclassified/unknown.

4.3.3 Functional Requirements

ML-01: Read network traffic in the form of numeric data.

ML-02: Determine whether the packets are anomalous or safe.

ML-03: Determine attack types based on the given data and provide a confidence score.

4.4 Model Adaptation/Calibration

4.4.1 Description and Priority

This function will establish and update the model to be adaptive to the network it's on. It will start by being connected to the network and then calibrated by taking a sample of network traffic. It

creates a baseline from this and allows the model to adapt to the network. If more devices are added or removed, then the system will be recalibrated to adapt to the new devices. It will refill the forest each time to accurately define what is normal and what isn't. It will display information on how it was built to show what data it used and information such as the number of devices to make sure the calibration looks correct. This is a high-priority feature as it is needed for our project to function properly.

4.4.2 Stimulus/Response Sequences

The model is connected to the network and starts the calibration process. Once completed, the model begins scanning the traffic for anomalies. This can be redone through the Web UI if the user chooses to do so. It should be recalibrated if devices are added or removed. A device count and training data information will be accessible to ensure the process is completed correctly.

4.4.3 Functional Requirements

AC-01: Model shall calibrate once prompted the first time it is connected to the network.

AC-02: Model shall recalibrate once prompted to adjust for a change in network devices.

AC-03: Model shall display the data used to create the baseline and show the connected devices.

4.5 Detection Logging and Storage

4.5.1 Description and Priority

This feature will record all detections found by the ML model and store them in the database running on the Raspberry Pi. This allows the user to review them at a later date through the WebUI. Without the logging and storage, there is no way to review detections to ensure that the results are accurate, and it makes it harder to fix the vulnerabilities. The database will be encrypted using SQLCipher to protect user data from leaking. This is medium priority since it isn't required for the project to work, but important for understanding

4.5.2 Stimulus/Response Sequences

Once flagged data is classified as anomalous, it is stored in the database and can be viewed from the WebUI. Relevant packet data is also logged with the alert to help the user track down and resolve the issue on their network. Data is removed from the database after 6 months.

4.5.3 Functional Requirements

DLS-01: Store anomalous packet data in the database on the Raspberry Pi

DLS-02: Shall be viewable through the WebUI.

DLS-03: Data shall be removed from the database after 6 months

DLS-04: Data stored shall be encrypted using SQLCipher.

4.6 Web-Based Dashboard

4.6.1 Description and Priority

The dashboard will be a locally hosted WebUI that will act as the interface between the user and the IDS (intrusion detection system). The dashboard will include a notification center, history page, performance page, and network information page. It will contain charts to visually represent the performance of the system. The model should be allowed to be calibrated on initial connection and recalibrated when devices are added or removed from the network using a button. It shall display the number of devices connected to the network and the data used to calibrate the model. It will have metrics to ensure the model was calibrated correctly and can be redone if necessary. The dashboard will also include the ability for the user to log in to protect their data from being viewed by anyone. Nothing will be visible until the code is entered, and it will be done through hashing. This is a medium-priority feature since it isn't needed for functionality but is important for viewing results.

4.6.2 Stimulus/Response Sequences

When the ML model deems a packet to be anomalous, the WebUI will receive a copy of the packet data from the database. Users will then view the notification in the notification dashboard where they can dismiss it. The notifications will remain on the dashboard until dismissed. The user can then view previously dismissed notifications on the history page. The user can also view the IDS performance as well as the network information. When the model is added to the network or devices on the network change, the system will need to be calibrated through the dashboard. It will show the devices connected to the network and the data used for calibration. There will be a login feature to be able to view the data and performance metrics. The user will be prompted to log in before anything can be viewed.

4.6.3 Functional Requirements

WBD-01: Retrieve anomalous packet data from database

WBD-02: Display anomalous packet notifications

WBD-03: Move notifications from dashboard to history when dismissed

WBD-04: Display performance and activity data

WBD-05: Display all IoT devices on the network

WBD-06: Allow model to be calibrated/recalibrated.

WBD-07: Display metrics on what data the model used to calibrate/recalibrate.

WBD-08: Web UI shall require user login through a password before any pages are loaded.

WBD-09: Password shall be stored using hashing.

WBD-10: Allow user to remove entries from the database.

4.7 Evaluation and Performance Modeling

4.7.1 Description and Priority

This feature will determine how the system's effectiveness is measured. The two models will be evaluated differently. The random forest will be based on metrics such as precision, recall, F1, and false-positive rate. The isolation forest will have those values computed as binary anomalous vs. normal classification, and then separately the random forest will be used to determine the attack type. This is due to the isolation forest not using labeled data, so the output will have to be checked manually for correctness. This is a high priority because of its necessity for determining if the project works.

4.7.2 Stimulus/Response Sequences

When a test using labeled data and the random forest is run, the system will generate precision, recall, F1 scores, and false positive rate. When the data comes from the network live, the detection latency, CPU usage, memory usage, and network throughput will be recorded. To test the adaptability between networks, it will be tested in different network simulations to see if the results are the same, and then it will be tested on data it wasn't trained on to see if it is able to flag it as anomalous or not.

4.7.3 Functional Requirements

EVAL-01: The system will report scoring metrics for each traffic class and each model.
EVAL-02: The system will measure and report detection latency, CPU usage, memory usage, and network throughput when running on the Raspberry Pi.
EVAL-03: The system will support being able to be tested on different networks to see how well it adapts.
EVAL-04: The system will support being able to be tested on data that was unseen initially to see how well it adapts.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

When running the machine learning model, the model must be able to keep up with the packets that are collected by Scapy. The software may be implemented with multithreading when checking packets if it is found that the pipeline slows down when the model is checking linearly. The pipeline must be able to make a decision in such a way that there is not a growing backup of packets for it to check. Because all traffic is read by the model, this is where a bottleneck may exist, so it must be able to run at the speed of packets coming through the network.

The WebUI must be able to refresh, showing the packets stored in the database at a speed that keeps up with the packet flow. This is of lower priority than the machine learning model, but still an important feature.

5.1.1 Performance Requirements

PERF-01: While the system is operating in real time, the Raspberry Pi shall measure and report performance metrics including: detection latency, CPU Utilization, memory utilization, network throughput, and packet/frame processing rate under a defined test workload.
PERF-02: The system performance acceptable thresholds for detection latency, CPU utilization, memory utilization, and sustainable packet/frame rate are TBD pending hardware integration and testing.

Once the hardware becomes available, testing will be conducted, and numerical values can be defined for metric target rates.

5.2 Safety Requirements

Network Guardian has an issue that calls for safety requirements. While doing research, if a malicious packet was improperly employed, there could be a localized breach which could cause loss of data. If there was a development error in the ML model, there could be user data loss from a lack of containment of abnormal

packets. If any of our simulated attacks were to escape the testing environment, it could create potential legal and regulatory issues under applicable FCC rules and the CFAA.

5.2.1 Safety Requirements

SAFE-01: Attacks shall only happen when the system is completely isolated from the internet.
SAFE-02: Attacks shall only be performed from team-owned computers.
SAFE-03: Attacks shall only be performed on the isolated network.
SAFE-04: Attacks shall only be performed when an attack plan is created, and the system is in the correct state.

5.3 Security Requirements

Network Guardian is a cybersecurity product and will therefore have security requirements. The database will be encrypted, and data retention should last six months. The database will be encrypted using SQLCipher. User data will be collected by default since the model will need to read the local network traffic to analyze and determine if the packet is anomalous. For our testing purposes, we will be using simulated data, so this only applies if the product is used on a real network.

User data should remain local to prevent any public data leak, even though the data will be simulated during testing. The IoT should remain on a LAN to localize access. To ensure that regular data will not be lost, the ML model will not block or discard actual network traffic.

5.4 Software Quality Attributes

The software should have multiple key attributes to ensure it is able to perform and capture what we are trying to achieve. Those qualities are included below with a brief explanation.

- Adaptive: The model must be able to adapt to different environments so we can test how well it will do when deployed. This will be done through our calibration/recallibration feature.
- Lightweight: Our system runs entirely locally, meaning everything must be able to run on our Raspberry Pi. This will be done by using lightweight software and minimizing the number of processes running at once.
- Accurate: The model should ensure results are consistent and accurate to ensure reliability in the product. This is measured through performance metrics such as recall, precision, f1 score, and false positives. The goal for the random forest model should be 90% or greater for F1 scores, and the isolation forest will aim for 80% or greater. The false positive rate should be less than 1% for both models.
- Secure: The data we collect to be stored in the database should be protected so personal information isn't leaked. This will be done through encrypting the database and keeping the dashboard locked behind a password.

5.5 Business Rules

Network Guardian will keep all users' information private within the Raspberry Pi. User information, as well as packet information, will not be stored externally. Research will be conducted directly on the LAN. All results from research will be based on what is seen in the network. Within the network, the user will have access to manage devices, view performance statistics, and view notifications for anomalous packets. All

anomalous packet data will be stored in the database, where it will remain for six months. Regular packets will not be stored. Packet storage data will follow the specifications in section 5.3: Security Requirements.

6. Other Requirements

The database will be needed to store the abnormal packet information. We will be storing the following attributes of each packet. In addition to storing the packet information, we will be storing data relating to the model's performance.

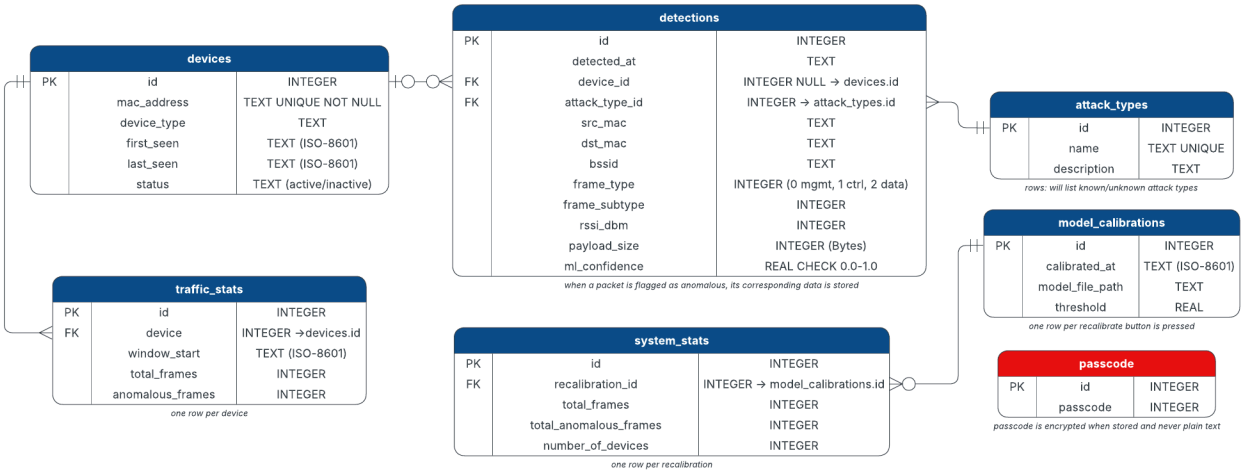


Figure 4: Database ER Diagram

Appendix A: Glossary

Address Resolution Protocol (ARP) Spoofing- an attack that forges address-resolution traffic to intercept or redirect communications between devices on a network
Aegean Wi-Fi Intrusion Dataset 2 (AWID2)- public dataset of labeled 802.11 wireless traffic
Anomalous- deviating from what is normal
Artificial intelligence (AI)- The ability of a computer to perform tasks that would usually require human intelligence; includes reasoning, pattern recognition, and decision-making
Broker- a system that receives messages from one device/application and forwards them to the appropriate device/application. In an IoT, it manages communication between devices.
Database- an organized collection of data that can be searched electronically
Deauthentication Attack- an attack that sends forged Wi-Fi management frames to forcibly disconnect a device from its network
Dynamic Host Configuration Protocol (DHCP)- a tool that automatically gives IP addresses and network settings to devices when they join the network
Graphical User Interface (GUI)- a visual way for users to interact with electronic devices through the use of buttons, icons, and windows
Hypertext Transfer Protocol (HTTP)- an application-layer communication protocol used to transfer hypermedia documents and load webpages
IEEE 802.11b/g/n- a set of standards for wireless networking defining how Wi-Fi devices exchange data over radio frequencies

Internet of Things (IoT)- network of physical devices connected to the internet that send, receive, and collect data
Internet Protocol (IP)- the network layer that has a set of rules responsible for addressing, routing, and delivering packets
Intrusion Detection System (IDS)- a cybersecurity system that detects malicious activity and reports it to the user
Isolation Forest- an unsupervised machine learning model that detects anomalies by measuring how easily a data point can be separated from the rest through random splits
Local Area Network (LAN)- a network of devices connected within a limited geographical area
Machine Learning (ML)- a type of AI where computers learn from data to make predictions without being explicitly programmed for all situations
Media Access Control (MAC) Address- the media access control address is the unique hardware identifier
Message Queuing Telemetry Transport (MQTT)- a lightweight communication protocol designed for IoT devices
Packet- a piece of information sent over a computer network
Packet Sniffer- a software tool that monitors, intercepts, and logs data traffic in a computer network
Random Forest- a supervised machine learning model made up of many decision trees. Each tree independently classifies a piece of data, and the outcome is based on the majority vote
Raspberry Pi- small, low-cost, single-board computer used for design, programming, and other computing applications
Received Signal Strength Indicator (RSSI)- measures how strong a wireless signal is
Replay attack- an attack that captures legitimate network traffic and resends it later to deceive a system
Transmission Control Protocol/Internet Protocol (TCP/IP)- set of protocols computers use to connect and send/receive data over networks
Web User Interface (WebUI)- a graphical interface that will allow users to interact with a software application through a web browser
WebSocket- a communication protocol that provides full-duplex communication over a TCP connection
Wi-Fi Protected Access 2-Advanced Encryption Standard (WPA2-AES)- a wireless network protocol that encrypts data sent over Wi-Fi

Appendix B: Analysis Models

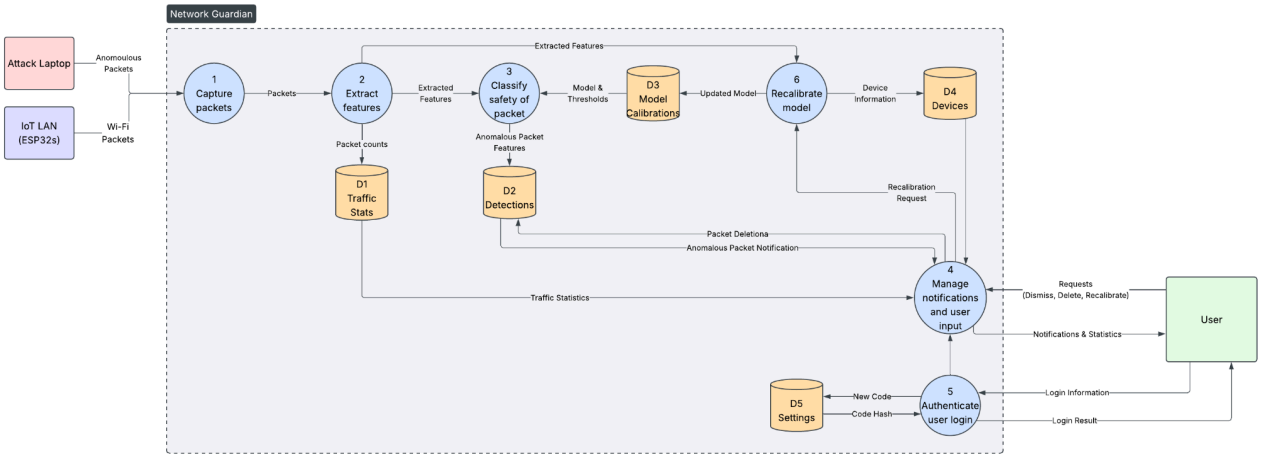


Figure 5: Data Flow Diagram

Appendix C: To Be Determined List

1. Finalize Software Versions
2. Refine WebUI design
3. Review Listed standards
4. Define User access
5. Inspect Research strategy
6. Evaluate Storage methods